In [1]:
```
import sys
from pathlib import Path

print("Python  :", sys.version.split()[0])
print("Folder  :", Path.cwd().name)

_missing = []
for _name in ['numpy', 'pandas', 'sklearn', 'fastapi', 'joblib', 'requests
    try:
        __import__(_name)
    except ImportError:
        _missing.append(_name)

for _name in ['numpy', 'pandas', 'sklearn', 'fastapi', 'joblib', 'requests
    _mark = "missing" if _name in _missing else "ok"
    print(f"  {_name:<14} {_mark}")

if _missing:
    print()
    print("STOP. Some libraries are missing:", ", ".join(_missing))
    print("Ask your instructor to run the setup in labs/SETUP.md.")
else:
    print()
    print("All good. You can carry on to Step 1.")
```

```
Python  : 3.14.8
Folder  : P08-docker
  numpy          ok
  pandas         ok
  sklearn        ok
  fastapi        ok
  joblib         ok
  requests       ok

All good. You can carry on to Step 1.
```

In [2]:
```
# The delivery-time dataset every practical in this course uses.
# If the file is missing we build it again from the same seed, so every
# student in the room gets byte-for-byte the same 600 rows.

import csv
from pathlib import Path

import numpy as np

SEED = 42
N_ROWS = 600
DATA = Path("..") / "data" / "delivery_times.csv"


def make_delivery_csv(path=DATA):
    """Write the 600-row delivery dataset. Same formula as the lectures."
    rng = np.random.default_rng(SEED)
    distance_km = np.round(rng.uniform(0.5, 12.0, N_ROWS), 2)
    prep_time_min = np.round(rng.uniform(5, 30, N_ROWS), 0)
```

```
    traffic_level = rng.integers(1, 4, N_ROWS)
    rain = rng.binomial(1, 0.25, N_ROWS)
    delivery_min = np.round(
        6.0
        + 3.1 * distance_km
        + 0.65 * prep_time_min
        + 4.2 * traffic_level
        + 5.5 * rain
        + rng.normal(0, 2.5, N_ROWS),
        1,
    )
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(["distance_km", "prep_time_min", "traffic_level",
                    "rain", "delivery_min"])
        for i in range(N_ROWS):
            w.writerow([distance_km[i], int(prep_time_min[i]),
                        int(traffic_level[i]), int(rain[i]), delivery_min[
    return path


if not DATA.exists():
    make_delivery_csv()
    print("dataset rebuilt ->", DATA)
else:
    print("dataset found   ->", DATA)
```

```
dataset found   -> ../data/delivery_times.csv
```

In [3]:

```
import shutil
import subprocess
from pathlib import Path

WORK = Path("work")
WORK.mkdir(exist_ok=True)

IMAGE = "delivery-api:1.0"
CONTAINER = "delivery-lab"
PORT = 8010

DOCKER_OK = False
if shutil.which("docker") is None:
    print("Docker is NOT installed on this machine.")
    print("Install Docker Desktop, then restart Jupyter.")
else:
    probe = subprocess.run(["docker", "version", "--format",
                            "{{.Server.Version}}"],
                           capture_output=True, text=True)
    if probe.returncode == 0 and probe.stdout.strip():
        DOCKER_OK = True
        print("Docker server version:", probe.stdout.strip())
    else:
        print("Docker is installed but not running.")
        print("Start Docker Desktop and wait for the whale icon")
        print("to stop animating, then run this cell again.")
```

```
print()
print("DOCKER_OK =", DOCKER_OK)
```

```
Docker server version: 29.9.1

DOCKER_OK = True
```

In [4]:
```
def docker(*args, timeout=900, quiet=False):
    """Run one docker command and show its output."""
    if not DOCKER_OK:
        print(f"SKIPPED (no Docker): docker {' '.join(args)}")
        return None
    print(f"$ docker {' '.join(args)}")
    try:
        done = subprocess.run(["docker", *args], timeout=timeout,
                              capture_output=True, text=True)
    except subprocess.TimeoutExpired:
        print(f"timed out after {timeout} seconds")
        return None
    output = (done.stdout + done.stderr).strip()
    if output and not quiet:
        print(output[-2500:])
    print("exit code:", done.returncode)
    print("-" * 60)
    return done


docker("--version")
```

```
$ docker --version
Docker version 29.9.1, build dfc520dcc2
exit code: 0
------------------------------------------------------------
```

Out[4]:
```
CompletedProcess(args=['docker', '--version'], returncode=0, stdout='Dock
er version 29.9.1, build dfc520dcc2\n', stderr='')
```

In [5]:
```
import joblib
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]
orders = pd.read_csv(DATA)
X_train, _, y_train, _ = train_test_split(
    orders[FEATURES], orders["delivery_min"],
    test_size=0.2, random_state=42,
)
joblib.dump(LinearRegression().fit(X_train, y_train),
            WORK / "model.joblib")

app_py = '''"The delivery-time prediction service."

from pathlib import Path

import joblib
```

```
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel, Field

HERE = Path(__file__).resolve().parent
FEATURES = ["distance_km", "prep_time_min", "traffic_level", "rain"]

app = FastAPI(title="Delivery Time Predictor", version="1.0.0")
model = joblib.load(HERE / "model.joblib")


class Order(BaseModel):
    distance_km: float = Field(..., gt=0, le=50)
    prep_time_min: int = Field(..., ge=0, le=120)
    traffic_level: int = Field(..., ge=1, le=3)
    rain: int = Field(..., ge=0, le=1)


@app.get("/health")
def health():
    return {"status": "ok", "model_loaded": model is not None}


@app.post("/predict")
def predict(order: Order):
    row = pd.DataFrame([order.model_dump()])[FEATURES]
    return {"delivery_min": round(float(model.predict(row)[0]), 1),
            "model_version": app.version}
'''
(WORK / "app.py").write_text(app_py, encoding="utf-8")

requirements = "\n".join([
    "fastapi==0.141.1",
    "uvicorn==0.52.1",
    "scikit-learn==1.9.0",
    "pandas==2.3.3",
    "joblib==1.5.3",
]) + "\n"
(WORK / "requirements.txt").write_text(requirements, encoding="utf-8")

for f in sorted(WORK.iterdir()):
    if f.is_file():
        print(f"  {f.name:<20} {f.stat().st_size:>9,} bytes")
```

```
 .dockerignore              65 bytes
 .dockerignore.mine         44 bytes
 Dockerfile                370 bytes
 app.py                    911 bytes
 model.joblib              967 bytes
 requirements.txt           81 bytes
```

In [6]:
```
dockerfile = """# The delivery-time prediction service.
FROM python:3.12-slim

WORKDIR /app
```

```
# Libraries first: this layer is cached and rarely changes.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Code and model last: these change often.
COPY app.py .
COPY model.joblib .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
"""

(WORK / "Dockerfile").write_text(dockerfile, encoding="utf-8")
print((WORK / "Dockerfile").read_text(encoding="utf-8"))
```

```
# The delivery-time prediction service.
FROM python:3.12-slim

WORKDIR /app

# Libraries first: this layer is cached and rarely changes.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Code and model last: these change often.
COPY app.py .
COPY model.joblib .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

In [7]:
```
dockerignore = """__pycache__/
*.pyc
.ipynb_checkpoints/
*.ipynb
.venv/
.git/
*.db
"""

(WORK / ".dockerignore").write_text(dockerignore, encoding="utf-8")
print((WORK / ".dockerignore").read_text(encoding="utf-8"))
```

```
__pycache__/
*.pyc
.ipynb_checkpoints/
*.ipynb
.venv/
.git/
*.db
```

In [8]:
```
build = docker("build", "-t", IMAGE, str(WORK))
```

```
$ docker build -t delivery-api:1.0 work
-3.7.0-py3-none-any.whl (26 kB)
#9 9.673 Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)
#9 9.705 Downloading typing_inspection-0.4.4-py3-none-any.whl (14 kB)
#9 9.740 Downloading tzdata-2026.5-py2.py3-none-any.whl (347 kB)
#9 9.788 Downloading annotated_types-0.8.0-py3-none-any.whl (13 kB)
#9 9.817 Downloading anyio-4.15.1-py3-none-any.whl (132 kB)
#9 9.861 Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
#9 9.891 Downloading idna-3.20-py3-none-any.whl (69 kB)
#9 10.09 Installing collected packages: pytz, tzdata, typing-extensions, th
readpoolctl, six, numpy, narwhals, joblib, idna, h11, click, annotated-type
s, annotated-doc, uvicorn, typing-inspection, scipy, python-dateutil, pydan
tic-core, anyio, starlette, scikit-learn, pydantic, pandas, fastapi
#9 19.31 Successfully installed annotated-doc-0.0.5 annotated-types-0.8.0 a
nyio-4.15.1 click-8.5.0 fastapi-0.141.1 h11-0.16.0 idna-3.20 joblib-1.5.3 n
arwhals-2.27.1 numpy-2.5.4 pandas-2.3.3 pydantic-2.14.1 pydantic-core-2.50.
1 python-dateutil-3.0.0 pytz-2026.5 scikit-learn-1.9.0 scipy-1.18.1 six-1.1
7.0 starlette-1.10.0 threadpoolctl-3.7.0 typing-extensions-4.16.0 typing-in
spection-0.4.4 tzdata-2026.5 uvicorn-0.52.1
#9 19.31 WARNING: Running pip as the 'root' user can result in broken permi
ssions and conflicting behaviour with the system package manager, possibly
rendering your system unusable. It is recommended to use a virtual environm
ent instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action
option if you know what you are doing and want to suppress this warning.
#9 19.60
#9 19.60 [notice] A new release of pip is available: 25.0.1 -> 26.2.2
#9 19.60 [notice] To update, run: pip install --upgrade pip
#9 DONE 20.4s

#10 [5/6] COPY app.py .
#10 DONE 0.1s

#11 [6/6] COPY model.joblib .
#11 DONE 0.1s

#12 exporting to image
#12 exporting layers
#12 exporting layers 12.1s done
#12 exporting manifest sha256:454dd19338414f342989ee047151793b91fb225d2f141
0e6e42fa36344d23470 0.0s done
#12 exporting config sha256:9da2683f43ce2a76fc506d0b1821603e8179b1aef14a7e1
fcac6867b765d2207 0.0s done
#12 exporting attestation manifest sha256:ae05283be14ecae8499449695f5e10f20
a05771491afc0d0cf72a35d365a8781 0.0s done
#12 exporting manifest list sha256:61ae98043d7e994f1fd4b7a47497926a7a219935
f8f48800af3b1b2d2dd60271 0.0s done
#12 naming to docker.io/library/delivery-api:1.0 done
#12 unpacking to docker.io/library/delivery-api:1.0
#12 unpacking to docker.io/library/delivery-api:1.0 3.5s done
#12 DONE 15.7s
exit code: 0
--------------------------------------------------------------
```

In [9]:
```
docker("images", IMAGE)
```

```
$ docker images delivery-api:1.0
IMAGE              ID             DISK USAGE   CONTENT SIZE   EXTRA
delivery-api:1.0   61ae98043d7e        676MB          158MB
exit code: 0
------------------------------------------------------------
```

Out[9]:
```
CompletedProcess(args=['docker', 'images', 'delivery-api:1.0'], returncod
e=0, stdout='IMAGE              ID             DISK USAGE   CONTENT SIZE
EXTRA\ndelivery-api:1.0   61ae98043d7e        676MB          158MB
\n', stderr='')
```

In [10]:
```python
import time

docker("rm", "-f", CONTAINER, quiet=True)      # clear any leftover
run = docker("run", "-d", "--rm", "--name", CONTAINER,
             "-p", f"{PORT}:8000", IMAGE)

if DOCKER_OK:
    time.sleep(6)          # give uvicorn a moment to start
    docker("ps", "--filter", f"name={CONTAINER}")
```

```
$ docker rm -f delivery-lab
exit code: 0
------------------------------------------------------------
$ docker run -d --rm --name delivery-lab -p 8010:8000 delivery-api:1.0
4d6209c14e0ae13de6f53f93b1e421414c1657f8c7c7e169043fcdc19f8ef60d
exit code: 0
------------------------------------------------------------
$ docker ps --filter name=delivery-lab
CONTAINER ID   IMAGE              COMMAND                  CREATED
STATUS         PORTS                                       NAMES
4d6209c14e0a   delivery-api:1.0   "uvicorn app:app --h…"   6 seconds ago
Up 6 seconds   0.0.0.0:8010->8000/tcp, [::]:8010->8000/tcp   delivery-lab
exit code: 0
------------------------------------------------------------
```

In [12]:
```python
import requests

if DOCKER_OK:
    base = f"http://127.0.0.1:{PORT}"
    try:
        health = requests.get(f"{base}/health", timeout=10)
        print("GET /health ->", health.status_code, health.json())

        order = {"distance_km": 5.0, "prep_time_min": 20,
                 "traffic_level": 2, "rain": 0}
        pred = requests.post(f"{base}/predict", json=order,
                             timeout=10)
        print("POST /predict ->", pred.status_code, pred.json())
    except requests.RequestException as problem:
        print("Could not reach the container:", problem)
        print("Run the previous cell again, wait, then retry.")
else:
    print("SKIPPED (no Docker): nothing is listening on port",
          PORT)
```

```
GET /health -> 200 {'status': 'ok', 'model_loaded': True}
POST /predict -> 200 {'delivery_min': 43.0, 'model_version': '1.0.0'}
```

In [13]:
```
docker("logs", "--tail", "15", CONTAINER)
docker("stop", CONTAINER)

print()
print("Containers still running:")
docker("ps")
```

```
$ docker logs --tail 15 delivery-lab
INFO:     172.17.0.1:60500 - "GET /health HTTP/1.1" 200 OK
INFO:     172.17.0.1:60510 - "POST /predict HTTP/1.1" 200 OK
/usr/local/lib/python3.12/site-packages/sklearn/base.py:525: InconsistentVe
rsionWarning: Trying to unpickle estimator LinearRegression from version 1.
9.1 when using version 1.9.0. This might lead to breaking code or invalid r
esults. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainabi
lity-limitations
  warnings.warn(
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
exit code: 0
------------------------------------------------------------
$ docker stop delivery-lab
delivery-lab
exit code: 0
------------------------------------------------------------

Containers still running:
$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
exit code: 0
------------------------------------------------------------
```

Out[13]:
```
CompletedProcess(args=['docker', 'ps'], returncode=0, stdout='CONTAINER I
D   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES\n', stderr
='')
```

In [14]:
```
# T1
T1_patterns = [
    "__pycache__/",
    "*.pyc",
    ".venv/",
    ".git/",
    "tests/",
    "*.md"
]

(WORK / ".dockerignore.mine").write_text("\n".join(T1_patterns) + "\n")

print(T1_patterns)
```

```
['__pycache__/', '*.pyc', '.venv/', '.git/', 'tests/', '*.md']
```

In [21]:
```
# T2
T2_fixed = """FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
COPY model.joblib .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
"""

print("YOURS:")
print(T2_fixed)
```

```
YOURS:
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
COPY model.joblib .

EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

In [23]:
```
# T3 — Rebuild and serve a rainy order

import time
import requests

T3_minutes = None

if not DOCKER_OK:
    print("SKIPPED: Docker is not running.")
else:
    # Write Dockerfile
    (WORK / "Dockerfile.v2").write_text(T2_fixed)

    # Build image
    subprocess.run([
        "docker", "build",
        "-f", str(WORK / "Dockerfile.v2"),
        "-t", "delivery-api:2.0",
        str(WORK)
    ], check=True)
```

```
    # Run container
    subprocess.run([
        "docker", "run", "-d",
        "--name", "delivery-lab-2",
        "-p", "8011:8000",
        "delivery-api:2.0"
    ], check=True)

    # Wait for startup
    time.sleep(3)

    # POST a rainy order
    response = requests.post(
        "http://127.0.0.1:8011/predict",
        json={
            "distance_km": 4,
            "prep_time_min": 30,
            "traffic_level": 3,
            "rain": 1
        },
        timeout=10
    )
    response.raise_for_status()

    result = response.json()
    T3_minutes = result.get(
        "delivery_min",
        result.get("delivery_minutes")
    )

    print("T3_minutes =", T3_minutes)
```

```
#0 building with "default" instance using docker driver

#1 [internal] load build definition from Dockerfile.v2
#1 transferring dockerfile: 269B done
#1 DONE 0.0s

#2 [auth] library/python:pull token for registry-1.docker.io
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.12-slim
#3 DONE 1.9s

#4 [internal] load .dockerignore
#4 transferring context: 105B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 95B done
#5 DONE 0.0s

#6 [1/6] FROM docker.io/library/python:3.12-slim@sha256:a0f57064cbdcd6285fa
45f51237ab0b35ea8ebaed06a1f5973e89e19dfb86704
#6 resolve docker.io/library/python:3.12-slim@sha256:a0f57064cbdcd6285fa45f
51237ab0b35ea8ebaed06a1f5973e89e19dfb86704 0.0s done
#6 DONE 0.0s

#7 [3/6] COPY requirements.txt .
#7 CACHED

#8 [4/6] RUN pip install --no-cache-dir -r requirements.txt
#8 CACHED

#9 [5/6] COPY app.py .
#9 CACHED

#10 [2/6] WORKDIR /app
#10 CACHED

#11 [6/6] COPY model.joblib .
#11 CACHED

#12 exporting to image
#12 exporting layers done
#12 exporting manifest sha256:454dd19338414f342989ee047151793b91fb225d2f141
0e6e42fa36344d23470 done
#12 exporting config sha256:9da2683f43ce2a76fc506d0b1821603e8179b1aef14a7e1
fcac6867b765d2207 done
#12 exporting attestation manifest sha256:fd285c1384409ec54e67ac7b750b77b69
f6915d3e099f9608975c16dfd87ef9b
#12 exporting attestation manifest sha256:fd285c1384409ec54e67ac7b750b77b69
f6915d3e099f9608975c16dfd87ef9b 0.0s done
#12 exporting manifest list sha256:9a6b1d5465820b732cd71fa274aa275d5d866314
7f1d54601d88a3a6408bf250 0.0s done
#12 naming to docker.io/library/delivery-api:2.0 done
#12 unpacking to docker.io/library/delivery-api:2.0 0.0s done
#12 DONE 0.1s
```

```
a844b9331ac2023a1d04b47cc933319accfc85d8a1f0a6aa298038a69e9e7e61
T3_minutes = 56.1
```

In [24]:

```python
# ----------------------------------------------------------------
# SELF-CHECK --- run this when you have attempted the tasks above.
# It never breaks your notebook. A task you have not done yet simply
# shows FAIL.
# ----------------------------------------------------------------

_results = []


def _check(label, fn):
    """Evaluate one graded condition without ever raising."""
    try:
        ok = bool(fn())
    except Exception:
        ok = False
    _results.append((label, ok))


_check('T1 | the file exists', lambda: (WORK / '.dockerignore.mine').is_fi
_check('T1 | it contains all six patterns, one per line', lambda: set(ln.s
_check('T1 | T1_patterns lists the same six', lambda: set(T1_patterns) ==
_check('T2 | the fixed Dockerfile sets a WORKDIR', lambda: 'WORKDIR' in T2
_check('T2 | requirements are installed before the code is copied', lambda
_check('T2 | uvicorn is told to listen on 0.0.0.0', lambda: '--host' in T2
_check('T2 | it still starts from a slim python base and exposes a port',
_check('T3 | the second Dockerfile was written', lambda: (WORK / 'Dockerfi
_check('T3 | the container returned a sensible prediction', lambda: 30 < f

print("================================================================")
print("SELF-CHECK   Practical 08 --- Packing the Service into a Container")
print("================================================================")
for _label, _ok in _results:
    print(f"  [{'PASS' if _ok else 'FAIL'}]  {_label}")
print("----------------------------------------------------------------")
_passed = sum(1 for _, _ok in _results if _ok)
print(f"  {_passed} of {len(_results)} checks passed")
print("================================================================")
if _passed == len(_results):
    print("Well done. Save the notebook and submit it.")
else:
    print("Read the FAIL lines above, fix those tasks, run this cell again
```

```
======================================================================
SELF-CHECK   Practical 08 --- Packing the Service into a Container
======================================================================
  [PASS]  T1 | the file exists
  [PASS]  T1 | it contains all six patterns, one per line
  [PASS]  T1 | T1_patterns lists the same six
  [PASS]  T2 | the fixed Dockerfile sets a WORKDIR
  [PASS]  T2 | requirements are installed before the code is copied
  [PASS]  T2 | uvicorn is told to listen on 0.0.0.0
  [PASS]  T2 | it still starts from a slim python base and exposes a port
  [PASS]  T3 | the second Dockerfile was written
  [PASS]  T3 | the container returned a sensible prediction
----------------------------------------------------------------------
  9 of 9 checks passed
======================================================================
Well done. Save the notebook and submit it.
```

In [ ]: